

MTL301 – Project Report

RRPRP Drilling Manipulator

Adarsh Panjwani, Kabeer More
Instructor: Dr. Saikat Sahu

November 23, 2025

Abstract

This report describes the design, kinematic solution, and Blender-based simulation of a 5-joint RRPRP drilling manipulator. The manipulator drills a grid of holes in a metal plate following the problem statement.

1 Problem statement

The task is to design and simulate an RRPRP manipulator that drills holes in a plate. The plate and target grid, and additional requirements, are given in the project brief.

2 Introduction to the problem

The manipulator has five joints in an RRPRP configuration (rotary, rotary, prismatic, rotary, prismatic). The goal is to reach a set of hole positions on a plate and perform a vertical drilling motion at each hole. Important constraints are that the end-effector must approach vertically for drilling and retract between holes. For modelling the problem, the approach used was to first model the RRP link in 2-D, and then model the remaining RP chain separately.

3 Short introduction to Blender used for this work

Blender is a free 3D modelling and animation tool. We used Blender to:

- Build simple link geometry for the manipulator,
- Create a controller object that stores joint variables as custom properties, to use for controlling the arm.
- Drive (manipulate) link transforms (translation/rotation) with drivers that read the controller properties
- Run a Python script to compute inverse kinematics and insert keyframes to animate the sequence.

Using Blender for robotics is useful because it lets us see the motion, test reachability, and make short animation clips for demonstration - and is open source - which helps us use different libraries if needed.

4 Creation of the model and workspace

4.1 Scene setup and units

Made sure Blender units are set to **centimeters (cm)** for consistency with the problem statement. In Blender preferences we set the unit scale so 1 Blender unit = 1 cm..

4.2 Objects and naming

The Python script expects an object named **Controller** (an Empty). This Controller holds custom properties: q_1 , q_2 , q_3 (angles) and d_1 , d_2 (distances). Link meshes are parented or driven so that changes to these properties move the robot correctly. The links are appropriately parented in the blender layers. Additionally, text objects are used to display the values of the controlled variables in the viewport.

4.3 Model Dimensions

Selecting the dimensions for the links was an important part of the project. We first defined the maximum reach required for the bot, which was decided based on the placement of the workspace. The dimensions used in the model (just the relevant ones) are:

- $l_1 = 14$ cm
- $l_2 = 12.5$ cm
- $d_2 = 3$ cm
- $l_3 = 10$ cm
- $l_4 = 14$ cm
- $l_5 = 10$ cm

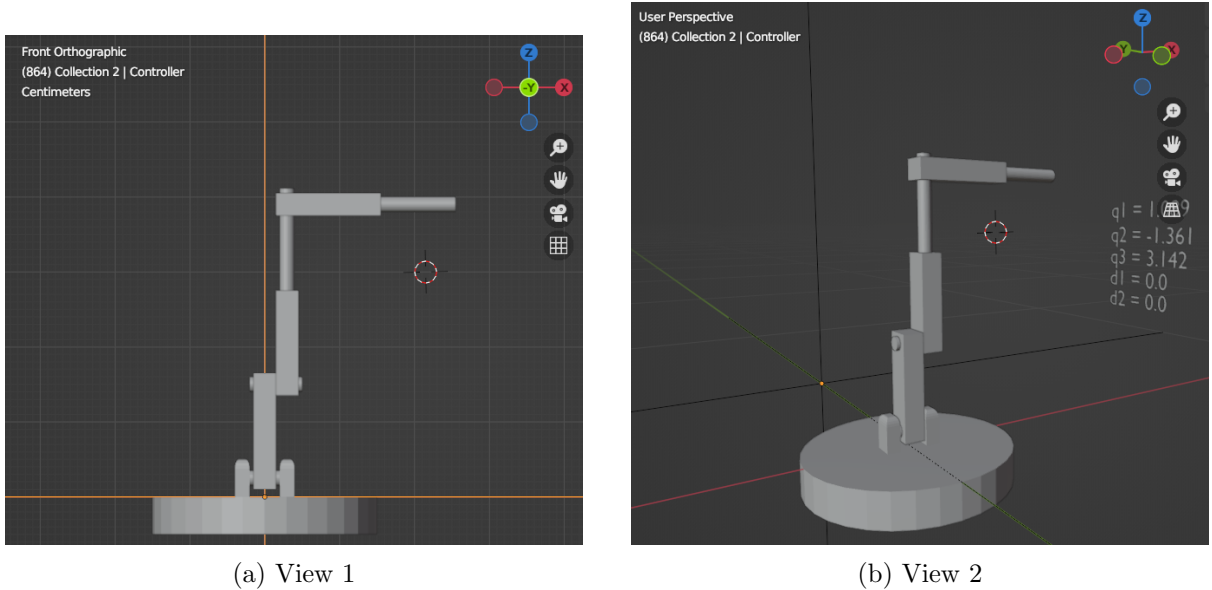


Figure 1: Two Blender views

These dimensions were selected to balance workspace coverage, structural stability, and smooth motion execution during drilling operations.

4.4 Drivers and keyframes

Each link transform (rotation or translation) has a driver using the Controller's custom properties. The script writes values in degrees for readability and converts angles to radians before storing in Blender (Blender drivers expect radians). The script then inserts keyframes on the Controller properties to animate the motion.

5 Kinematic model and inverse kinematics in the 2D plane

5.1 Model simplification

The inverse kinematics is solved in the y - z plane treating the relevant portion of the manipulator as a two-link system: a fixed first link of length a_1 and an effective second link of variable length L_2 . The second link includes a base offset a_2 and an extension (prismatic) of up to a maximum m . The input target coordinates for the IK are (x, y, z) where in this project $x = \pm 1$ is used to pick one of two mirrored cycles and y, z come from the targets list (given in cm).

5.2 Mathematical steps

Let the planar distance be

$$R = \sqrt{y^2 + z^2}.$$

The effective second-link length L_2 must satisfy mechanical bounds:

$$L_{2,\min} = \max(|R - a_1|, a_2), \quad L_{2,\max} = \min(R + a_1, a_2 + m).$$

If $L_{2,\min} > L_{2,\max}$ there is no feasible solution for that target.

We choose the smallest feasible $L_2 = L_{2,\min}$ as the solution in this project. Using the law of cosines we compute the second joint angle q_2 by

$$\cos q_2 = \frac{R^2 - a_1^2 - L_2^2}{2a_1L_2},$$

and then

$$q_2 = \text{sign}(y) \arccos(\cos q_2),$$

where the sign of q_2 is chosen based on the sign of y to fix elbow orientation.

The base rotation q_1 is computed through standard 2-link geometry as

$$\phi = \text{atan2}(z, y), \quad \psi = \text{atan2}(-L_2 \sin q_2, a_1 + L_2 \cos q_2),$$

$$q_1 = \phi - \psi.$$

The prismatic extension is

$$d_1 = L_2 - a_2.$$

Finally, the end-effector rotation q_3 is chosen as either 0° or 180° depending on the sign of x (this enforces a vertical approach, mirrored for the two cycles).

All returned angles are converted to degrees in the script for readability; the script then converts to radians before writing to Blender properties.

5.3 Drilling cycle and stroke limits

Before drilling, the robot aligns the tool using the revolute joint q_3 (the fourth joint in the RRRP chain). This joint controls the orientation of the drill bit, ensuring it remains perpendicular to the metal plate surface before the drilling stroke begins.

The full drilling sequence is as follows:

1. **Approach:** Move the end-effector above the target hole using (q_1, q_2, d_1) .
2. **Orientation:** Adjust q_3 so that the drill axis is perpendicular to the workpiece.
3. **Pre-drill:** Keep the drill fully retracted (set $d_2 = 0$).
4. **Drilling:** Extend the drill stroke ($d_2 > 0$), up to its maximum allowable value (e.g., 10 units).
5. **Retract:** Return the drill to the safe position ($d_2 \leftarrow 0$).
6. **Transition:** Move to the next hole in the sequence.

6 Explanation of the Python functions

All code is written in a single script used inside Blender's scripting workspace. An overview of the main functions follows.

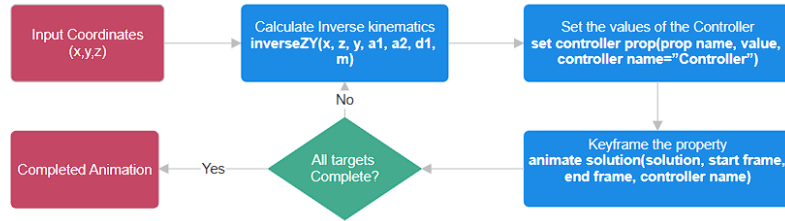


Figure 2: Logic Flow

6.1 inverseZY(x, z, y, a1, a2, d1, m)

This function computes the inverse kinematics for a planar target in the y-z plane. Inputs are the target coordinates and link geometry parameters. It returns a dictionary with keys: L2, q2, q1, d1, q3 or None if no solution exists. The value of q_3 is decided by whether the target x coordinate is positive or not. The value of d_2 is not calculated, rather it is used as the drilling motion of the robotic arm.

The function follows the math shown in the previous section. It clips cosine values to the $[-1, 1]$ range to avoid numerical errors by python and handles the sign choice for q_2 based on y .

6.2 set_controller_prop(prop_name, value, controller_name="Controller")

This helper writes a named property (angle or distance) into the Controller object safely. It clamps the value to the decided range for each link. for example, the value of q_2 is restricted to -160d to 160d. It converts angles from degrees to radians, and updates Blender's _RNA_UI metadata.

6.3 `animate_solution(solution, start_frame, end_frame, controller_name)`

Given a solution (dictionary from `inverseZY`), this function writes the Controller properties and inserts keyframes to animate the approach, drilling stroke, and retract. The keyframe pattern used in the project is:

- **start_frame**: record current pose (baseline), set `d2=0` and keyframe.
- **end_frame - 10**: set target pose (approach complete) and keyframe.
- **end_frame - 5**: set `d2 = 0.1` (drill stroke) and keyframe.
- **end_frame**: set `d2 = 0` and keyframe (reset).

This pattern models approach, short drill action, retraction, and reset for the next hole.

7 Targets, units and animation timing

The project uses a target list. All units in the script and Blender scene are centimeters (cm). The code runs two cycles: first for $x = +1$ (one side) and then for $x = -1$ (mirrored) as requested.

Each target is animated over a 30-frame motion window and the script increments the frame counter by 35 frames between targets. For a Blender frame rate of 24 fps, this corresponds to approximately 1.46 seconds per hole. Running two cycles means the whole animation plays both left and right sequences in order.

8 Limitation and possible improvements

The current work is not a proper model of a 4DOF robotic arm (considering the last link as just the drilling motion). Since this was the first time modelling, and figuring out blender workflow, the project is a bit naive, but now that the initial learning curve is conquered, the following improvements can be implemented

- Using a complete DH parameter model and transformation matrixes for the path planning.
- Analyzing the dynamics of the model as well (velocity, inertia atc.)
- Extended to analyzing drilling power required, and forces on the model